```
annotated_cheatsheet =
"""================================================================================
OPENCV COMPLETE FUNCTION & PARAMETER CHEATSHEET (PYTHON)
================================================================================

1. LOADING, DISPLAYING & SAVING IMAGES
--------------------------------------------------------------------------------
import cv2 as cv
import numpy as np

# cv.imread(filename, flags)
# - filename: Path to the image file (e.g., 'image.jpg')
# - flags: (Optional) How to read image. Options:
#     cv.IMREAD_COLOR (1) -> Default, loads 3-channel BGR image.
#     cv.IMREAD_GRAYSCALE (0) -> Loads image as single-channel grayscale.
#     cv.IMREAD_UNCHANGED (-1) -> Loads image including alpha (transparency)
channel.
img = cv.imread('path.jpg', cv.IMREAD_COLOR)

# cv.imshow(winname, mat)
# - winname: Name of the GUI window displaying the image.
# - mat: Image array (numpy ndarray) to display.
cv.imshow('Image Window', img)

# cv.waitKey(delay)
# - delay: Time in milliseconds to wait for keyboard input.
#     0 -> Waits indefinitely until a key is pressed.
#     >0 -> Waits for specified ms (e.g., 20ms). Returns ASCII value of key
pressed.
cv.waitKey(0)

# cv.destroyAllWindows()
# - Closes all active OpenCV GUI windows created by cv.imshow.
cv.destroyAllWindows()

# cv.imwrite(filename, img)
# - filename: Output path/file name (e.g., 'output.jpg').
# - img: Image array to save.
cv.imwrite('output.jpg', img)

# cv.resize(src, dsize, fx, fy, interpolation)
# - src: Input image array.
# - dsize: Output dimensions tuple (width, height) in pixels.
# - fx, fy: (Optional) Scale factors along X and Y axes if dsize is (0, 0).
# - interpolation: Algorithm used for pixel resampling:
#     cv.INTER_LINEAR -> Default, good for zooming/scaling.
#     cv.INTER_AREA -> Best for shrinking (downsampling) images.
#     cv.INTER_CUBIC -> Slow, higher quality for enlarging.
resized = cv.resize(img, (640, 480), interpolation=cv.INTER_LINEAR)
```

```
# cv.cvtColor(src, code)
# - src: Input image array.
# - code: Color space conversion code (e.g., cv.COLOR_BGR2GRAY, cv.COLOR_BGR2HSV,
cv.COLOR_BGR2RGB).
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
# Mathematical Formula:
# Gray Pixel Value = (0.299 * Red) + (0.587 * Green) + (0.114 * Blue)


2. DRAWING & ANNOTATIONS
--------------------------------------------------------------------------------
# OpenCV uses BGR (Blue, Green, Red) tuples for colors, e.g., (255, 0, 0) = Blue.

# cv.line(img, pt1, pt2, color, thickness, lineType)
# - img: Target image on which to draw.
# - pt1: Starting point tuple (x1, y1) in pixels.
# - pt2: Ending point tuple (x2, y2) in pixels.
# - color: BGR color tuple, e.g., (0, 255, 0) for Green.
# - thickness: Line width in pixels (default=1).
# - lineType: (Optional) cv.LINE_AA for anti-aliased (smooth) lines.
cv.line(img, (0, 0), (200, 200), (0, 255, 0), thickness=2, lineType=cv.LINE_AA)

# cv.rectangle(img, pt1, pt2, color, thickness)
# - pt1: Top-left corner tuple (x1, y1).
# - pt2: Bottom-right corner tuple (x2, y2).
# - thickness: Line thickness. Use -1 or cv.FILLED to draw a solid filled
rectangle.
cv.rectangle(img, (50, 50), (250, 250), (255, 0, 0), thickness=-1)

# cv.circle(img, center, radius, color, thickness)
# - center: Center point tuple (center_x, center_y).
# - radius: Circle radius in pixels.
# - thickness: Line thickness (-1 to fill).
cv.circle(img, (150, 150), 50, (0, 0, 255), thickness=3)

# cv.putText(img, text, org, fontFace, fontScale, color, thickness, lineType)
# - text: String text to overlay.
# - org: Bottom-left origin point tuple (x, y) where text starts.
# - fontFace: Font type (e.g., cv.FONT_HERSHEY_SIMPLEX, cv.FONT_HERSHEY_COMPLEX).
# - fontScale: Font scale factor (multiplier for size).
# - color: BGR tuple, e.g., (255, 255, 255) for White.
# - thickness: Text character outline thickness.
cv.putText(img, "OpenCV", (10, 300), cv.FONT_HERSHEY_SIMPLEX, 1.0, (255, 255, 255),
2)


3. IMAGE BLURRING & NOISE REDUCTION
--------------------------------------------------------------------------------
# cv.blur(src, ksize) [Average / Box Filter]
# - ksize: Tuple defining kernel size (width, height), e.g., (5, 5). Replaces each
```

```
pixel
#   with the average value of surrounding pixels in a 5x5 window.
avg_blur = cv.blur(img, (5, 5))

# cv.GaussianBlur(src, ksize, sigmaX, sigmaY)
# - ksize: Kernel size tuple (w, h). BOTH NUMBERS MUST BE ODD (1, 3, 5, 7, ...).
# - sigmaX: Gaussian kernel standard deviation along X-axis. 0 = auto-calculated
from ksize.
# - sigmaY: Gaussian kernel standard deviation along Y-axis. Default=0.
# Formula: G(x,y) = (1 / (2 * pi * sigma^2)) * exp(-(x^2 + y^2) / (2 * sigma^2))
gblur = cv.GaussianBlur(img, (5, 5), sigmaX=0)

# cv.medianBlur(src, ksize)
# - ksize: Aperture linear size. MUST BE AN ODD INTEGER > 1 (e.g., 3, 5, 7).
# - Excellent for removing "salt and pepper" noise while keeping sharp edges.
mblur = cv.medianBlur(img, 5)

# cv.bilateralFilter(src, d, sigmaColor, sigmaSpace)
# - d: Diameter of each pixel neighborhood used during filtering.
# - sigmaColor: Filter sigma in color space. Larger value means farther colors get
mixed.
# - sigmaSpace: Filter sigma in coordinate space. Larger value means farther pixels
influence each other.
# - Preserves sharp edges while smoothing flat areas.
bfilter = cv.bilateralFilter(img, d=9, sigmaColor=75, sigmaSpace=75)


4. THRESHOLDING & BINARIZATION
--------------------------------------------------------------------------------
# cv.threshold(src, thresh, maxval, type)
# - src: Grayscale input image.
# - thresh: Threshold limit value (0-255).
# - maxval: Maximum value assigned if pixel passes threshold (usually 255).
# - type: Binarization type:
#     cv.THRESH_BINARY -> if pixel > thresh then maxval else 0
#     cv.THRESH_BINARY_INV -> if pixel > thresh then 0 else maxval
#     cv.THRESH_OTSU -> Calculates optimal thresh automatically (pass thresh=0).
# Returns: (ret, thresh_img) tuple where 'ret' is the computed threshold value.
ret, thresh = cv.threshold(gray, 127, 255, cv.THRESH_BINARY)
ret_otsu, otsu = cv.threshold(gray, 0, 255, cv.THRESH_BINARY + cv.THRESH_OTSU)

# cv.adaptiveThreshold(src, maxValue, adaptiveMethod, thresholdType, blockSize, C)
# - maxValue: Value assigned if pixel condition is met (usually 255).
# - adaptiveMethod: How local threshold is calculated:
#     cv.ADAPTIVE_THRESH_MEAN_C -> Mean of local neighborhood minus C.
#     cv.ADAPTIVE_THRESH_GAUSSIAN_C -> Weighted sum of local neighborhood minus C.
# - thresholdType: Must be cv.THRESH_BINARY or cv.THRESH_BINARY_INV.
# - blockSize: Size of pixel neighborhood used to calculate local thresh. MUST BE
ODD (e.g., 11).
# - C: Constant subtracted from the mean/weighted mean (e.g., 2). Fine-tunes
```

```
thresholding.
adaptive = cv.adaptiveThreshold(gray, 255, cv.ADAPTIVE_THRESH_GAUSSIAN_C,
cv.THRESH_BINARY, 11, 2)


5. GRADIENTS & EDGE DETECTION
--------------------------------------------------------------------------------
# cv.Canny(image, threshold1, threshold2, apertureSize, L2gradient)
# - image: Grayscale input image.
# - threshold1: Lower bound hysteresis threshold (e.g., 50). Edges below this are
rejected.
# - threshold2: Upper bound hysteresis threshold (e.g., 150). Edges above this are
kept.
#   Rule of thumb: threshold2 is typically 2x or 3x threshold1.
# - apertureSize: (Optional) Kernel size for Sobel operator internal calculation
(default=3).
edges = cv.Canny(gray, 50, 150)

# cv.Sobel(src, ddepth, dx, dy, ksize)
# - ddepth: Output image depth. cv.CV_64F prevents overflow (negative gradients).
# - dx: Order of derivative x (1 = vertical edge detection, 0 = ignore).
# - dy: Order of derivative y (1 = horizontal edge detection, 0 = ignore).
# - ksize: Size of extended Sobel kernel (1, 3, 5, or 7).
sobelx = cv.Sobel(gray, cv.CV_64F, dx=1, dy=0, ksize=3)
sobely = cv.Sobel(gray, cv.CV_64F, dx=0, dy=1, ksize=3)
# Magnitude Formula: Magnitude = sqrt(sobelx^2 + sobely^2)


6. MORPHOLOGICAL OPERATIONS
--------------------------------------------------------------------------------
# Structuring Element / Kernel Creation
# np.ones((height, width), dtype): Creates a matrix of ones.
kernel = np.ones((5, 5), np.uint8)

# cv.dilate(src, kernel, iterations)
# - src: Binary image.
# - kernel: Structuring element defining shape/size.
# - iterations: Number of times dilation is applied. Expands white foreground
areas.
dilated = cv.dilate(thresh, kernel, iterations=1)

# cv.erode(src, kernel, iterations)
# - Erodes/shrinks white foreground areas. Useful for removing isolated noise.
eroded = cv.erode(thresh, kernel, iterations=1)

# cv.morphologyEx(src, op, kernel)
# - op: Morphological operation type:
#     cv.MORPH_OPEN -> Erosion followed by Dilation (Removes external background
noise).
#     cv.MORPH_CLOSE -> Dilation followed by Erosion (Fills internal holes in
```

```
foreground).
#     cv.MORPH_GRADIENT -> Dilation minus Erosion (Outlines object boundaries).
opening = cv.morphologyEx(thresh, cv.MORPH_OPEN, kernel)
closing = cv.morphologyEx(thresh, cv.MORPH_CLOSE, kernel)


7. GEOMETRIC TRANSFORMATIONS
--------------------------------------------------------------------------------
# Translation (Shifting position)
# Matrix M = [[1, 0, tx], [0, 1, ty]]
# - tx: Horizontal shift in pixels (Positive = shift right, Negative = shift left).
# - ty: Vertical shift in pixels (Positive = shift down, Negative = shift up).
M_trans = np.float32([[1, 0, 50], [0, 1, 100]]) # Shift right 50px, down 100px
# cv.warpAffine(src, M, dsize)
# - M: Transformation matrix (2x3 array).
# - dsize: Output image dimensions (width, height).
translated = cv.warpAffine(img, M_trans, (640, 480))

# Rotation
# cv.getRotationMatrix2D(center, angle, scale)
# - center: Rotation center point tuple (cx, cy).
# - angle: Rotation angle in degrees (Positive = counter-clockwise, Negative =
clockwise).
# - scale: Isotropic scale factor (1.0 = original size, 0.5 = half size, 2.0 =
double size).
M_rot = cv.getRotationMatrix2D((320, 240), angle=45, scale=1.0)
rotated = cv.warpAffine(img, M_rot, (640, 480))

# Flip
# cv.flip(src, flipCode)
# - flipCode:
#      0 -> Flip vertically around X-axis.
#      1 -> Flip horizontally around Y-axis (mirror effect).
#     -1 -> Flip both vertically and horizontally.
flipped = cv.flip(img, 1)


8. CONTOURS & SHAPE ANALYSIS
--------------------------------------------------------------------------------
# cv.findContours(image, mode, method)
# - image: Single-channel binary image (e.g., thresholded image).
# - mode: Contour retrieval mode:
#     cv.RETR_EXTERNAL -> Retrieves only extreme outer contours.
#     cv.RETR_LIST -> Retrieves all contours without hierarchy.
#     cv.RETR_TREE -> Retrieves all contours and reconstructs full hierarchy of
nested contours.
# - method: Contour approximation method:
#     cv.CHAIN_APPROX_NONE -> Stores all contour points (uses more memory).
#     cv.CHAIN_APPROX_SIMPLE -> Compresses horizontal, vertical, and diagonal
segments (stores only endpoint coordinates).
```

```
# Returns: (contours, hierarchy) tuple.
contours, hierarchy = cv.findContours(thresh, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)

# cv.drawContours(image, contours, contourIdx, color, thickness)
# - contours: Python list of contours.
# - contourIdx: Index of contour to draw (-1 draws ALL contours in the list).
# - color: BGR color tuple.
# - thickness: Line thickness (-1 to fill interior).
cv.drawContours(img, contours, -1, (0, 255, 0), thickness=2)

cnt = contours[0] # Take first contour

# cv.contourArea(contour)
# - Returns total calculated pixel area enclosed by contour.
area = cv.contourArea(cnt)

# cv.arcLength(curve, closed)
# - curve: Input contour.
# - closed: Boolean indicating if curve is closed (True) or open (False).
perimeter = cv.arcLength(cnt, True)

# cv.boundingRect(array)
# - Calculates minimal up-right bounding box for a contour.
# Returns: (x, y, w, h) -> Top-left point (x, y), Width (w), Height (h).
x, y, w, h = cv.boundingRect(cnt)

# cv.moments(array)
# - Calculates all spatial moments up to 3rd order for a contour/shape.
# - Returns dictionary of moment values ('m00', 'm10', 'm01', etc.).
# Mass/Area: M['m00']
# Centroid X Formula: cx = int(M['m10'] / M['m00'])
# Centroid Y Formula: cy = int(M['m01'] / M['m00'])
M = cv.moments(cnt)
if M['m00'] != 0:
    cx = int(M['m10'] / M['m00'])
    cy = int(M['m01'] / M['m00'])


9. COLOR DETECTION (HSV COLOR SPACE)
--------------------------------------------------------------------------------
# HSV: Hue (0-179 in OpenCV), Saturation (0-255), Value/Brightness (0-255)
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)

# Array definition: np.array([Hue, Saturation, Value], dtype=np.uint8)
# - Lower bound: Minimum acceptable H, S, V values.
# - Upper bound: Maximum acceptable H, S, V values.
lower_red = np.array([0, 120, 70])
upper_red = np.array([10, 255, 255])

# cv.inRange(src, lowerb, upperb)
```

```
# - Checks if array elements lie between two scalar limits.
# Returns binary mask (255 for pixels inside range, 0 for outside).
mask = cv.inRange(hsv, lower_red, upper_red)

# cv.bitwise_and(src1, src2, mask)
# - Calculates bitwise AND of two arrays pixel by pixel.
# - mask: (Optional) Operation is performed only on non-zero mask pixels.
res = cv.bitwise_and(img, img, mask=mask)


10. VIDEO PROCESSING & WEBCAM CAPTURE
--------------------------------------------------------------------------------
# cv.VideoCapture(index_or_filename)
# - index_or_filename: Device index (0 for primary webcam, 1 for external USB
camera) OR video filepath string ('video.mp4').
cap = cv.VideoCapture(0)

# cap.isOpened(): Checks if video file or camera stream opened successfully.
while cap.isOpened():
    # cap.read(): Grabs, decodes and returns next frame.
    # - ret: Boolean (True if frame retrieved successfully, False if stream ended).
    # - frame: Image array for current video frame.
    ret, frame = cap.read()

    if not ret:
        break

    cv.imshow('Webcam Stream', frame)

    # cv.waitKey(20) & 0xFF == ord('q')
    # - 20: Wait 20 milliseconds (~50 FPS).
    #   FPS Formula: Delay (ms) = 1000 / Target FPS (e.g., 1000 / 30 fps ≈ 33ms)
    # - & 0xFF: Bitwise mask to extract lower 8-bits of key code (cross-platform
fix).
    # - ord('q'): Returns ASCII value of key 'q'. Quits loop if 'q' is pressed.
    if cv.waitKey(20) & 0xFF == ord('q'):
        break

# cap.release(): Closes webcam feed or video file completely from system memory.
cap.release()
cv.destroyAllWindows()
================================================================================
"""

file_path = "opencv_annotated_cheatsheet.txt"
with open(file_path, "w", encoding="utf-8") as f:
    f.write(annotated_cheatsheet)

print(f"File created successfully: {file_path}")
```